

CSA0604-DESIGN AND ANALYSIS OF ALGORITHM

NAME : SANTHIYA S

REG NO : 192511360

1. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

· Input: $m=2, n=2, N=2, i=0, j=0$ · Output: 6

· Input: $m=1, n=3, N=3, i=0, j=1$ · Output: 12

CODE:

```
#include <stdio.h>

int m, n, N;

int dp[50][50][50];

int countWays(int i, int j, int steps)
{
    // Ball has moved outside the grid
    if (i < 0 || i >= m || j < 0 || j >= n)
        return 1;

    // Exactly N steps used but ball is still inside
    if (steps == N)
        return 0;

    // Already calculated
    if (dp[i][j][steps] != -1)
        return dp[i][j][steps];

    // Move in 4 directions
```

```

    int up = countWays(i - 1, j, steps + 1);

    int down = countWays(i + 1, j, steps + 1);

    int left = countWays(i, j - 1, steps + 1);

    int right = countWays(i, j + 1, steps + 1);

    dp[i][j][steps] = up + down + left + right;

    return dp[i][j][steps];
}

int main()
{
    int i, j;

    printf("Enter m, n, N: ");

    scanf("%d %d %d", &m, &n, &N);

    printf("Enter starting position (i, j): ");

    scanf("%d %d", &i, &j);

    // Initialize DP array with -1

    for (int a = 0; a < m; a++)

        for (int b = 0; b < n; b++)

            for (int c = 0; c <= N; c++)

                dp[a][b][c] = -1;

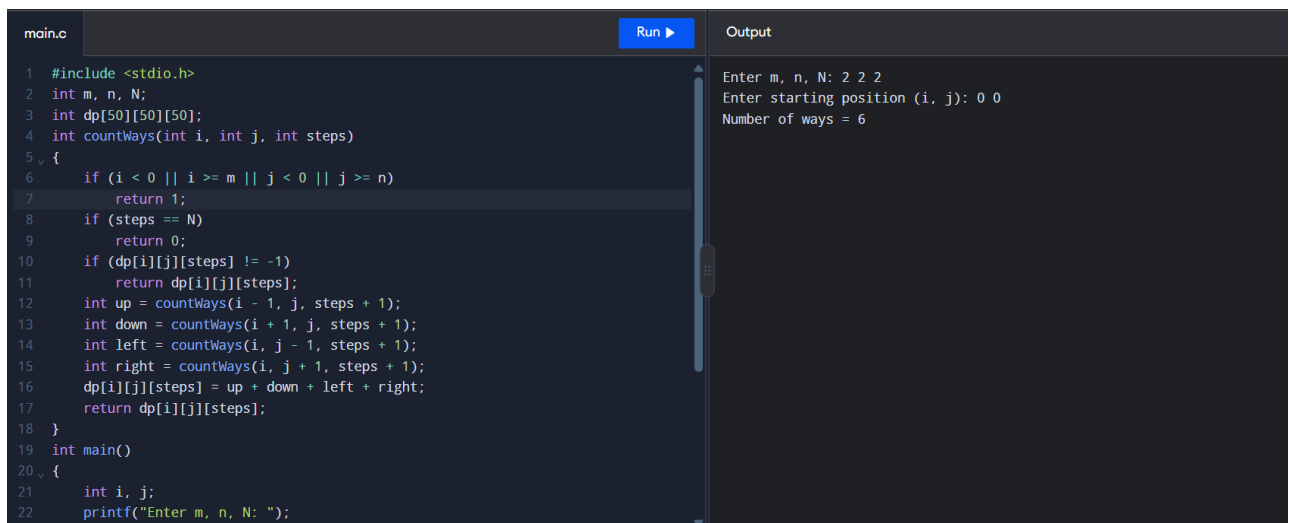
    int result = countWays(i, j, 0);

    printf("Number of ways = %d\n", result);

    return 0;
}

```

OUTPUT:



```
main.c Run ▶ Output
1 #include <stdio.h>
2 int m, n, N;
3 int dp[50][50][50];
4 int countWays(int i, int j, int steps)
5 {
6     if (i < 0 || i >= m || j < 0 || j >= n)
7         return 1;
8     if (steps == N)
9         return 0;
10    if (dp[i][j][steps] != -1)
11        return dp[i][j][steps];
12    int up = countWays(i - 1, j, steps + 1);
13    int down = countWays(i + 1, j, steps + 1);
14    int left = countWays(i, j - 1, steps + 1);
15    int right = countWays(i, j + 1, steps + 1);
16    dp[i][j][steps] = up + down + left + right;
17    return dp[i][j][steps];
18 }
19 int main()
20 {
21     int i, j;
22     printf("Enter m, n, N: ");
    Enter m, n, N: 2 2 2
    Enter starting position (i, j): 0 0
    Number of ways = 6
```

2. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

CODE:

```
#include <stdio.h>
```

```
int max(int a, int b)
```

```
{
```

```

        return (a > b) ? a : b;
    }

    // Find maximum money from houses between start and end
    int robLinear(int nums[], int start, int end)
    {
        int prev2 = 0;

        int prev1 = 0;

        for (int i = start; i <= end; i++)
        {
            int current = max(prev1, prev2 + nums[i]);

            prev2 = prev1;

            prev1 = current;
        }

        return prev1;
    }

    int rob(int nums[], int n)
    {
        // Only one house
        if (n == 1)
            return nums[0];

        // Case 1: Don't rob the last house
        int case1 = robLinear(nums, 0, n - 2);

        // Case 2: Don't rob the first house
        int case2 = robLinear(nums, 1, n - 1);
    }

```

```

        return max(case1, case2);
    }

int main()
{
    int n;

    printf("Enter number of houses: ");

    scanf("%d", &n);

    int nums[n];

    printf("Enter money in each house:\n");

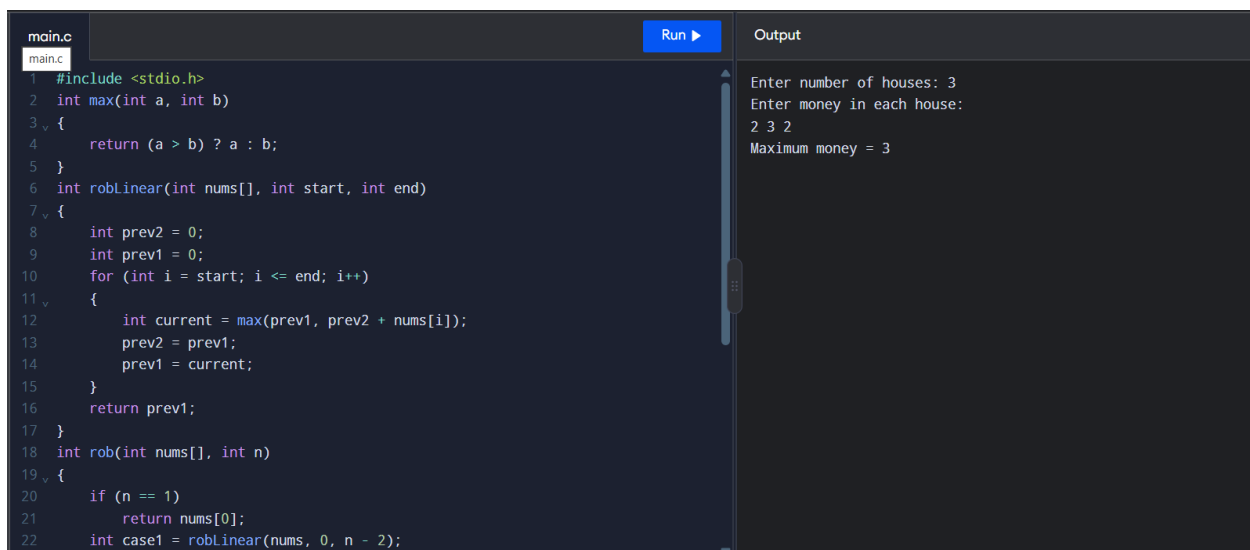
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    printf("Maximum money = %d\n", rob(nums, n));

    return 0;
}

```

OUTPUT:



The screenshot shows a C++ IDE with a file named 'main.c'. The code is as follows:

```

1 #include <stdio.h>
2 int max(int a, int b)
3 {
4     return (a > b) ? a : b;
5 }
6 int roblinear(int nums[], int start, int end)
7 {
8     int prev2 = 0;
9     int prev1 = 0;
10    for (int i = start; i <= end; i++)
11    {
12        int current = max(prev1, prev2 + nums[i]);
13        prev2 = prev1;
14        prev1 = current;
15    }
16    return prev1;
17 }
18 int rob(int nums[], int n)
19 {
20     if (n == 1)
21         return nums[0];
22     int case1 = roblinear(nums, 0, n - 2);

```

The output of the program is displayed on the right side of the IDE:

```

Enter number of houses: 3
Enter money in each house:
2 3 2
Maximum money = 3

```

3.You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: n=4 Output: 5

Input: n=3 Output: 3

CODE:

```
#include <stdio.h>

int climbStairs(int n)
{
    if (n <= 2)
        return n;

    int a = 1; // ways to reach step 0
    int b = 2; // ways to reach step 2
    int c;

    for (int i = 3; i <= n; i++)
    {
        c = a + b;
        a = b;
        b = c;
    }

    return b;
}

int main()
{
    int n;
```

```

printf("Enter number of steps: ");

scanf("%d", &n);

printf("Number of distinct ways = %d\n", climbStairs(n));

return 0;

}

```

OUTPUT:

The screenshot shows a C program named 'main.c' being executed. The code defines a function 'climbStairs' that uses a sliding window of two variables, 'a' and 'b', to calculate the number of distinct ways to climb 'n' stairs. The 'main' function prompts the user for the number of steps, which is 4. The output shows that there are 5 distinct ways to climb 4 stairs.

```

main.c
1  #include <stdio.h>
2  int climbStairs(int n)
3  {
4      if (n <= 2)
5          return n;
6      int a = 1;
7      int b = 2;
8      int c;
9      for (int i = 3; i <= n; i++)
10     {
11         c = a + b;
12         a = b;
13         b = c;
14     }
15     return b;
16 }
17 int main()
18 {
19     int n;
20     printf("Enter number of steps: ");
21     scanf("%d", &n);
22     printf("Number of distinct ways = %d\n", climbStairs(n));

```

Run ▶

Output

Enter number of steps: 4
Number of distinct ways = 5

4.A robot is located at the top-left corner of a $m \times n$ grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7, n=3$ Output: 28

Input: $m=3, n=2$ Output: 3

CODE:

```

#include <stdio.h>

int uniquePaths(int m, int n)

{

    int dp[m][n];

```

```

// First row = 1 way
for (int i = 0; i < m; i++)

    dp[i][0] = 1;

// First column = 1 way
for (int j = 0; j < n; j++)

    dp[0][j] = 1;

// Calculate remaining cells
for (int i = 1; i < m; i++)
{
    for (int j = 1; j < n; j++)

        {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
}

return dp[m - 1][n - 1];
}

int main()
{
    int m, n;

    printf("Enter m and n: ");

    scanf("%d %d", &m, &n);

    printf("Number of unique paths = %d\n", uniquePaths(m, n));

    return 0;
}

```

OUTPUT:

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int uniquePaths(int m, int n) 3 { 4 int dp[m][n]; 5 for (int i = 0; i < m; i++) 6 dp[i][0] = 1; 7 for (int j = 0; j < n; j++) 8 dp[0][j] = 1; 9 for (int i = 1; i < m; i++) 10 { 11 for (int j = 1; j < n; j++) 12 { 13 dp[i][j] = dp[i - 1][j] + dp[i][j - 1]; 14 } 15 } 16 return dp[m - 1][n - 1]; 17 } 18 int main() 19 { 20 int m, n; 21 printf("Enter m and n: "); 22 scanf("%d %d", &m, &n);</pre>		<pre>Enter m and n: 7 3 Number of unique paths = 28</pre>